Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Vize notları ve geç gönderilen kağıtlar hakkında açıklamalar.
- Örnek bir yazılım gereksinim spesifikasyonu (SRS) üzerinden doğru gereksinim belgesinin yapısı.
- Bir proje grubunun SRS belgesinin derste incelenmesi.
- Gereksinim analizinin amacı: problemi anlamak.
- Gereksinim mühendisliği (requirements engineering) adımları: başlangıç, gereksinim toplama, ayrıntılandırma, pazarlık, spesifikasyon, geçerleme, yönetim.
- Use case senaryolarında dikkat edilecekler.
- Proje gruplarının ve konularının kontrolü, sunum takvimi.
- Projede istenen testler: birim testi, sistem testi, kullanıcı testi.
- CMMI ve People CMM tekrarı.
- Çevik yaklaşımlar, XP ve eşli programlama (pair programming).
- Statik ve dinamik kalite değerlendirmesi.
- Test güdümlü yazılım geliştirme (TDD) tekrarı.
- Sonraki haftanın konuları: konfigürasyon ve sürüm yönetimi, bakım, yeniden mühendislik, tersine mühendislik.

Hocanın Özellikle Vurguladığı Kısımlar

- Grup üyelerinin hepsi aynı notu almayacak. Notu, projeye kimin ne kadar sahip çıktığına göre farklı olacak.
- Grubun her üyesi gönderilen belgeyi anlamış olmalı. Sunumda her üyeye soru sorulabilir.
- Vize kağıtlarında hataları bile aynı olan gruplar var. Bu kağıtlar daha düşük notlandırılacak.
- Sonradan mail ile gönderenler daha düşük baremden değerlendirilecek.
- SRS'te use case kullanmak zorunlu değil. Fonksiyonel gereksinimler net maddeler hâlinde de yazılabilir. Büyük sistemlerde modül modül ayrılabilir.
- SRS, gereksinim mühendisliğinin bütün adımlarından geçtikten sonra yazılan belgedir.
- Spesifikasyon yazıldıktan sonra mutlaka geçerleme yapılmalıdır.
- Use case'lerde alternatif senaryolar ve beklenmeyen durumlar da düşünülmelidir.
- Projede istenen testler:
  - Bir birim testi.
  - Bir sistem testi: fonksiyonel olmayan gereksinimler için. Açık kaynaklı bir araç seçilmeli (güvenlik, performans vb.).
  - Bir kullanıcı testi: fonksiyonel gereksinimlerin kullanıcı bakış açısıyla nasıl sağlandığı. SRS'e göre yapılmalı.
- Bütünlük testi şart değil. Tek bir modül içinde de olsa sistem testi yapılmalı.
- Eşli programlama XP'ye özgüdür. Diğer çevik yöntemlerde böyle bir zorunluluk yok.
- Statik kalite değerlendirmesine örnek: kod gözden geçirme (code review) ve inceleme (inspection).

Kısa Tekrar Notları

- Problemi anlamak için sorulacaklar: problem neyi çözüyor, çözümü kim kullanacak, bilinmeyenler neler, hangi veri ve işlevler gerekli, problem küçük parçalara bölünebilir mi,

grafik bir analiz modeli kurulabilir mi?
- Gereksinim mühendisliği adımları:
  - başlangıç (inception),
  - gereksinim toplama (elicitation),
  - ayrıntılandırma (elaboration),
  - pazarlık (negotiation),
  - spesifikasyon (specification),
  - geçerleme (validation),
  - gereksinim yönetimi (requirements management).
- Geçerlemede bakılanlar: açıklaştırılması gereken yerler, eksik bilgi (missing information), alt parçalar arasında uyumsuzluk, gerçekçi olmayan veya çelişen gereksinimler.
- Use case için: birincil ve ikincil aktörler, aktörlerin hedefleri, ön koşullar, aktörlerin işleri, alternatif yollar, beklenmeyen durumlar.
- CMMI'de ilk sertifikalandırılan düzey 2. düzeydir (Managed).
- 4. düzey: ölçümler arşivlenir ve yeni projelerin iş gücü, maliyet ve süre tahmininde kullanılır.
- 5. düzey: kurumsal yenilenme ve yaygınlaştırma, nedensel analiz, yeniliklerin koordinasyonu.
- TDD: gereksinim belirlenir belirlenmez test senaryosu planlanır; kod bu senaryoyu geçecek şekilde yazılır.

Detaylı Açıklamalar

Vize ve Notlar

- Hoca vize kağıtlarını okudu. Notları henüz not sistemine girmedi ve ilan etmedi.
- Ham olarak notların çoğu kötü değil.
- Değerlendirmede devam durumuna da bakılacak.
- Sınavda boş görünen kağıtlar oldu. Bir öğrenci farklı işletim sistemi ve ofis programı kullandığı için dosyanın açılmamış olabileceğini söyledi. Hoca dosyanın yeniden gönderilmesini istedi.

Örnek SRS Belgesinin Yapısı

- Hoca daha önce de incelenen, başka bir öğrenci projesine ait SRS örneğini yeniden gösterdi.
- İçerik:
  - İçindekiler.
  - Belgenin amacı ve özeti. Küçük bir proje için bu bölüm çok gerekli değil.
  - Ürünün tanımı: seçilen yazılım ne amaçla ve hangi işleri yapıyor? Bir use case diyagramıyla gösterilebilir.
  - Fonksiyonel gereksinimler: sistemin aktörleri, sistem sınırları, sistemin yaptığı işler.
  - Use case'ler için her senaryonun ayrıntılı belgelendiği tanımlar.
  - İşlevsel olmayan gereksinimler: kullanıcı arayüzü, kullanılabilirlik, güvenlik.
- Use case belgesi yerine temel akıştaki (basic path) her ana işlem tek madde olarak da yazılabilir.
- Birçok grup use case kullanmış. Modüllere ayırmış; yeni kayıt girişi, kaydın güncellenmesi ve alternatif yollar gibi işlemleri belirtmiş.
- İşlevsel gereksinimler adım adım yazılmalıdır.

Bir Grubun SRS Belgesinin İncelenmesi

- Hoca rastgele seçtiği bir grubun SRS belgesini açtı.
- Belgede giriş, amaç ve kapsam gibi tanımlamalar ile genel sistem bilgisi vardı.
- Grup fonksiyonel gereksinimleri use case diyagramlarıyla tanımlamıştı.
- Derste gruptan yalnız bir üye vardı. Belgeyi ağırlıkla başka bir üyenin hazırladığını söyledi.
- Hoca bunu sunumlar için bir uyarı olarak kullandı: herkes yapılan işi anlamalı ve anlatabilmeli.

Gereksinim Analizinin Amacı

- Gereksinim analizi öncelikle problemi anlamak için yapılır.
- Bu aşamadan sonra yazılımın temel alt parçaları belirlenir. Bunlar ileride modül olarak karşımıza çıkar.
- Projede öğrenciler hazır bir yazılım seçti ve gereksinim belgesini sonradan yazdı. Bu, tersten bir akıştır.
- Düz akışta, verilen bir problemi anlamak için önce gereksinim analizi yapılır.
- Örnek: Software Engineering kitabındaki SafeHome ev güvenlik sistemi. Sistem hangi işi yapıyor, nasıl bir ortamda çalışıyor?

Gereksinim Mühendisliği Adımları

- Başlangıç (inception): temel sorular sorulur.
  - Temel problemi anladık mı?
  - Çözümü isteyenler ve kullanıcılar kimler?
  - Çözümün yapısı ne olmalı?
  - Müşteri ile geliştirici arasındaki iletişim nasıl sağlanacak?
- Gereksinim toplama (elicitation): tüm paydaşlardan gereksinimler toplanır.
  - Paydaşlar: kullanıcılar, sponsor, yöneticiler, uç kullanıcılar.
  - Her paydaştan kendi yaptığı işi anlatması istenir.
  - Örnek: satış sipariş uygulamasında yalnız satış bölümü değil, siparişi alanlar, faturayı hazırlayanlar ve depo da ilgilidir.
- Ayrıntılandırma (elaboration): toplanan bilgilerle sistemin modeli çıkarılır.
  - Grup olarak use case diyagramı veya veri akış diyagramı gibi bir model seçilir.
  - Hedef: sisteme giren veriyi, işlemleri ve davranışları modellemek.
- Pazarlık (negotiation) iki anlam taşır:
  - Farklı tarafların hepsi çizilen modelin kendi işlerini doğru gösterdiğinde hemfikir mi?
  - Kullanıcı isteklerini anlayan geliştirici, bunları yazılıma dökebileceği konusunda kullanıcıyla anlaşabiliyor mu?
- Spesifikasyon (specification): SRS belgesi yazılır.
  - Modeller oluşturulur. Matematiksel bir formül varsa ifadesi geliştirilir.
  - Use case'ler gözden geçirilerek yayımlanır.
  - Gerekirse çalışır hâlde bir prototip çıkarılır.
- Geçerleme (validation): SRS üzerinde bir gözden geçirme mekanizmasıdır.
  - Sistem doğru yorumlandı mı?
  - Bazı bölümler daha açık ifade edilmeli mi?
  - Eksik bilgi var mı?
  - Büyük sistemlerde alt modüller arasında uyumsuzluk var mı?
  - Gerçekçi olmayan, başarılamayacak veya birbiriyle çelişen gereksinimler var mı? Pazarlık doğru yapıldıysa olmamalıdır.
- Gereksinim yönetimi: bütün bu adımların doğru planlanıp yürütülmesi; çalışanların ve kullanıcıların doğru birlikte çalışması.
- Hoca, bu adımların Yazılım Mühendisliği dersinde sık sorulan bir bölüm olduğunu hatırlattı.

Use Case Senaryolarında Dikkat Edilecekler

- Senaryolar bir sistem kullanımını anlatır. Her senaryoda önce aktör belirlenir.
- Sorulacak sorular:
  - Birincil aktör kim? İkincil aktör kim?
  - Aktörlerin hedefleri neler?
  - Senaryo başlamadan önce sağlanması gereken ön koşullar neler?
  - Her aktörün yapacağı işler neler?
  - Tanımlanan işler genişletilebilir mi? Alternatif yollar var mı?
  - Her aktör sistemde neyi isteyecek, üretecek veya değiştirecek?
  - Beklenmeyen durumlarda ne yapılacak?

Proje Grupları ve Sunum Takvimi

- Hoca proje grupları dosyasını paylaştı. Gruplarla isimleri ve konuları tek tek kontrol etti.
- Örnek proje konuları: seyahat acentesi sistemi, otopark otomasyonu, bilgisayar mühendisliği proje yönetim sistemi, kütüphane yönetim bilgi sistemi.
- Konusunu belirtmeyen ve SRS göndermeyen gruplar uyarıldı. Tek başına çalışan öğrenciler de var.
- Toplam 15 grup var. 13. haftada 7 veya 8 grup sunum yapacak.
- 13. haftada sunum yapmak isteyen gruplar bir sonraki derse kadar hocaya bildirecek. Hoca dosyada sunum takvimini oluşturacak.
- Proje teslimi:
  - 12. hafta dersinden bir iki gün sonra sistem üzerinden yapılacak.
  - Her gruptan bir temsilci yükleyecek. Grup üyelerinin isimleri doğru yazılmalı.
  - Hoca bunun için bir ödev tanımı açacak.
- Teslimden sonra 13. veya 14. haftada sunmak fark etmez.
- Test planı ve test raporları için ayrı format verilmeyecek. Test planındaki maddeler derste görüldü. Hoca projede hangi belgeleri istediğini anlatan bir belge paylaşacak.

Projede İstenen Testler

Test	İçerik
Birim testi	En az bir birim testi
Sistem testi	Fonksiyonel olmayan bir gereksinim için; açık kaynaklı bir araçla; güvenlik veya performans gibi bir konu seçilebilir
Kullanıcı testi	Fonksiyonel gereksinimlerin kullanıcı bakış açısıyla nasıl sağlandığı; SRS'e dayanır

- Sistem testi ve kullanıcı testi için hazırlanan SRS belgeleri kullanılacak.
- Grup içinde görev paylaşımı yaparak uygun aracı araştırın.

CMMI Tekrarı

- CMMI kalite iyileştirme konusunda yazılım geliştirme sürecini iyileştirmek için yapılması gerekenleri tanımlar.
- Hoca bu konunun vize sonrası işlenen kısım olduğunu belirtti.
- Sertifika sistemleri: CMMI, SPICE ve ISO tanımları.
- Süreç iyileştirme döngüsü: ölçüm, analiz, değişim, yeniden ölçüm.
  - CMMI 2 sertifikası alan kurum belli bir süre sonra 3'e çıkabilir veya 2'yi koruyamayabilir.
  - Benzer şekilde MÜDEK veya ABET akreditasyonu alan bir bölüm beş yıl sonra yeniden değerlendirilir.
- CMM'den farkı: CMMI'de proje yönetimi, mühendislik ve süreç yönetiminin hepsi bütünleşik olarak sertifikasyondan geçer. Kurum birinde 2., birinde 3. düzeyde olamaz.
- Düzeyler:
  - Başlangıç: gelişigüzel ve kaotik; planlar yapılsa da terk edilebilir.
  - Yönetilen: sertifikalandırılan ilk düzey; proje yönetimi kontrolleri, gereksinim yönetimi.
  - Tanımlı: süreçler standartlar, prosedürler, araçlar ve metotlarla anlatılmıştır; kurum bazında tutarlıdır; mutlaka bir süreç modeli kullanılır.
  - Sayısal Yönetilen: alt süreçler istatistiksel tekniklerle kontrol edilir; işler gerçek verilere dayanarak tahmin edilir.
  - En İyileştiren: sayısal geri besleme ve teknolojiyle sürekli iyileştirme; 4. düzeyden farkı süreçlerin değişme derecesi ve sürekli yeniliktir.
- 2. düzeyden itibaren ölçüm yapılır. 4. düzeyde yönetim bu ölçümlere dayanır.

Anahtar Süreç Alanları ve Yazılım Mühendisliği Aktiviteleri

- Yazılım mühendisliğinde temel aktiviteler: analiz, tasarım, gerçekleştirme, test.

- Destek aktiviteleri: konfigürasyon yönetimi, risk yönetimi, proje yönetimi, proje izleme ve kontrol.
- Düzeylere göre aktiviteler:
  - 2. düzey: gereksinim yönetimi, proje planlama, proje izleme ve kontrol, tedarikçi sözleşme yönetimi, ölçüm, konfigürasyon yönetimi. Her proje için ayrı yapılır.
  - 3. düzey: kurumsal standartlaşma. Gereksinim geliştirme, ürün entegrasyonu, doğrulama ve onaylama, kurumsal süreç tanımlama ve odaklanma, kurumsal eğitim, bütünleşik proje yönetimi, risk yönetimi.
  - 4. düzey: kurumsal süreç başarımı, sayısal proje yönetimi.
  - 5. düzey: kurumsal yenilenme ve yaygınlaştırma, nedensel analiz, yeniliklerin koordinasyonu.
- Destek süreç alanları:
  - Temel: ölçümleme ve analiz, kalite sağlama, konfigürasyon yönetimi.
  - İleri: nedensel analiz ve çözümleme, karar analizi, kurumsal bütünleştirme altyapısı.

People CMM Tekrarı

- People CMM, Software Engineering Institute tarafından geliştirildi. Hocaya göre şu anda CMMI'nin içindedir.
- Amaç: yazılım geliştirme yeteneğini anlamak, iş gücü yeteneklerini artırmak, kurum ve çalışan hedeflerini birlikte tutmak.
- Düzeyler:
  - Tekrarlanabilir: tazminat, eğitim, performans yönetimi, personel alımı, çalışma ortamı. Kurum bunları ispat edebilmelidir.
  - Tanımlı: katılımcı kültür, kariyer geliştirme, yetenek geliştirme, iş gücü planlama.
  - Yönetilen: kurumsal performans ayarlama, ekip temelli pratikler, takım kurma, danışmanlık.
  - En İyileşen: inovasyon ve yaratıcılık; personel açısından koçluk ve kişisel yetenek geliştirme.
- Sertifikasyon yalnız yazılım evleri için değildir. Bir şirketin içindeki yazılım birimi de alabilir.
  - Hoca örnek olarak Siemens'in yazılım birimini verdi. Son yıllarda 5. düzeye çıkmış olabileceğini, emin olmadan söyledi.

Çevik Yaklaşımlar ve XP

- Çevik yaklaşımlar artan müşteri taleplerini daha hızlı karşılamak için geleneksel modellere alternatif olarak çıktı.
- Örnek modeller: XP, Scrum, Kanban.
- XP'nin dört temel değeri: basitlik, iletişim, geri besleme, cesaret.
- Ortak özellikler:
  - küçük ve kısa aralıklı sürümler,
  - basit tasarım ve devamlı yeniden tasarım (refactoring),
  - ortak kod mülkiyeti,
  - sürekli tümleştirme (continuous integration),
  - haftada 40 saat, devam ettirilebilir hız,
  - müşterinin ekip üyesi olması (Scrum, XP ve Kanban'da),
  - kodlama standartları.
- Eşli programlama (pair programming):
  - XP'nin temel özelliğidir.
  - Çevik yöntemler 2000'li yılların başında tanıtıldığında ilk kullanılan XP idi. Eşli programlama en çok burada kullanıldı.
  - Ayrı bir test grubu yerine geliştirirken yanında bir kişi daha bulunur.
  - Bunun için tek ekran ve iki klavyeli çalışma ortamları çıktı.

Statik ve Dinamik Kalite Değerlendirmesi

- Çevik modellerde dinamik kalite değerlendirmesi (test) statik değerlendirmeden daha çok kullanılır.

- Statik kalite değerlendirmesine örnek: kod gözden geçirme (code review) ve inceleme (inspection).
- Çevik modellerde kalite aktiviteleri daha sık yapılır.
- Kullanıcı hikâyeleri, birim testler, müşteri geri beslemesi ve süreklilik çevik modellerin farkıdır.

Test Güdümlü Yazılım Geliştirme Tekrarı

- TDD çevik metotlarla birlikte kullanılır.
- Küçük ve hızlı iterasyonlarla, araçlarla hazırlanmış testler yazılır.
- Normal akışta gereksinim analizinde use case diyagramları veya başka modellerle gereksinimler belirlenir.
- TDD'de gereksinim belirlenir belirlenmez "bu gereksinimi hangi senaryoyla test ederim?" sorusu üzerinde çalışılır.
- Akış:
  - Use case diyagramı ve gereksinim senaryosu oluşturulur.
  - Bu gereksinimi sağlayacak test senaryosu planlanır.
  - Birim test case'leri yazılır ve kod olmadığı için çalıştırıldığında başarısız olduğu görülür.
  - Kod yazılır, test case'lerin geçmesi sağlanır.
  - Hata varsa kod yeniden düzenlenir.
- Hakan Erdoğmuş'un çalışmasına göre TDD programcı üretkenliğini artırdı. Kalitede çok fark görülmedi ama hız kazandırdı.
- Bu yüzden çevik yöntemler ve TDD şirketlerde çok kullanım alanı buldu.

Sonraki Haftalar

- 11. hafta: konfigürasyon yönetimi ve sürüm yönetimi (release management) birlikte.
  - Konfigürasyon yönetimi daha önce yalnız tanım olarak görüldü ve vizede soruldu.
- Bakım yönetimi.
- Yeniden mühendislik (software re-engineering) ve tersine mühendislik (reverse engineering): hangisi hangisinin içinde?
- 11. ve 12. haftalarda ders var. 13. ve 14. haftalarda proje sunumları yapılacak.

Ders sonrası notlar

- Altyazıda gereksinim mühendisliği adımları "inception, illustration, elaboration, negotiation" diye geçer. "Illustration" Whisper hatasıdır; doğrusu elicitation (gereksinim toplama).
- Hoca bir yerde çevik yaklaşımların genel adının XP olduğunu söyledi. Genel ad çevik (agile) yöntemlerdir; XP bunlardan biridir.
- Hoca basitlik, iletişim, geri besleme ve cesaret değerlerinin bütün çevik modellerde bulunduğunu söyledi. Bu dört değer XP'ye aittir. XP'nin ikinci sürümünde beşinci değer olarak saygı (respect) eklenmiştir. Çevik Manifesto'nun dört değeri farklıdır.
- Altyazıda "test driving development" ve "test drive on development" geçer. Doğrusu Test Driven Development'tır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.